



MPI-BASED 3D ANISOTROPIC RTM WITH FLETCHER'S METHOD AND GHOST CELL HALO EXCHANGE

Vinícius Daniel Spadotto, Lucas Mello Schnorr

SBAC-PAD 2026 @ Session #7 2026-10-15 – Madrid, Spain,

AGENDA

1. Motivation & Contributions
2. Methods & Materials
3. Results
4. Conclusion

Motivation & Contributions

- Halo exchange is the foundational technique for distributing iterative stencil computations across HPC nodes.
- Each MPI process keeps a *ghost-cell* layer — a local copy of its neighbors' boundary data — so it can **overlap** those transfers with its own interior computation.
- Whether this overlap genuinely **masks** communication depends on three interacting factors: the problem size, the partitioning topology, and the interconnection bandwidth.
- With small domains or slow networks, communication dominates; with large domains and fast networks, computation can fully hide the transfers.

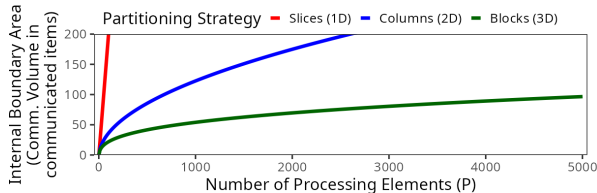
- Reverse Time Migration (RTM) reconstructs subsurface images by cross-correlating seismic wavefields propagated forward and backward in time.
- Fletcher's method solves the pseudo-acoustic wave equations that model how those waves travel through direction-dependent (anisotropic, TTI) rock.
- At every time step it couples a pressure field p with an auxiliary field q using an eighth-order finite-difference stencil.
- The method is computationally intensive and memory-bound, which is exactly why it benefits so strongly from GPU acceleration.

THE GAP IN THE LITERATURE

- Every prior GPU-accelerated Fletcher implementation runs on a **single node**, using shared memory or peer-to-peer GPU links.
- This single-node limit caps both the computational throughput and the GPU memory available as problems grow.
- The underlying distributed-stencil techniques — 3D block decomposition, asynchronous overlap, SimGrid calibration — are already well established.
- However, **no prior work has distributed the Fletcher method across multiple cluster nodes** via MPI with ghost-cell halo exchange.
- This leaves open how partitioning topology and network bandwidth *together* determine how effectively communication can be overlapped.

1. We present a **distributed MPI implementation** of Fletcher that uses 3D block decomposition and asynchronous ghost-cell communication, validated numerically against the single-node reference.
2. We add **GPU acceleration via CUDA**, keeping the wavefields resident in device memory while preserving the same overlap strategy as the CPU version.
3. We **evaluate on real HPC clusters** and show that bandwidth and topology jointly govern masking, from $\approx 20\%$ on a 1Gbps GPU cluster to over 87% on a 10Gbps CPU cluster.
4. We **use emulation** (SimGrid SMPI) to estimate the network capacity required to hide communication completely.

Methods & Materials



- The stencil does identical work everywhere, so the load is balanced and a **static** decomposition is enough.
- The domain can be split into **slices** $\{P, 1, 1\}$, **columns**, or **blocks**.
- **Blocks** win by minimizing the surface-to-volume ratio: slice boundaries grow as $2(P-1)L^2$, blocks only as $6(\sqrt[3]{P}-1)L^2$.
- We build the grid with `MPI_Dims_create` and `MPI_Cart_create`.
- **Caveat**: when P is prime, the grid collapses back to a slice layout.

Within each time step, every process performs five steps:

1. It posts non-blocking receives (`MPI_Irecv`) for the halos it expects from its neighbors.
2. It computes the **boundary strips** from the previous iteration and sends them asynchronously (`MPI_Isend`).
3. While those messages travel, it computes the **interior** of its subdomain.
4. It calls `MPI_Waitall` and inserts the halos that have arrived.
5. Finally, it swaps the wavefield buffers for the next step.

This only masks communication when the interior computation takes longer than the halo transfer. If the network is the bottleneck, step 4 stalls and communication dominates.

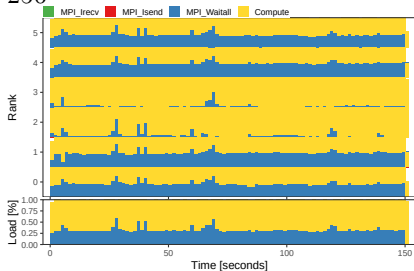
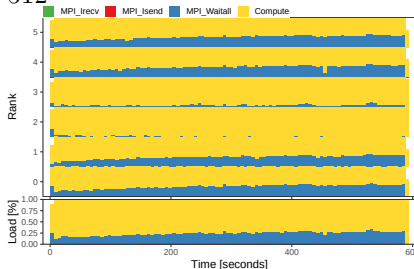
- The application follows a **coordinator-worker** model on a 3D Cartesian topology.
- The coordinator (rank 0) partitions the domain and sends only **compact metadata** — never the wavefield data itself.
- Each worker allocates and initializes its own sub-volume $(p_{\text{past}}, p_{\text{curr}}, q_{\text{past}}, q_{\text{curr}})$, so there is **no central memory bottleneck**.
- Output is collected in a streaming fashion, avoiding a second bottleneck at the end of the run.
- Every process exchanges halos with its 26 neighbors, for both wavefield components p and q .
- On multi-GPU nodes, each process is bound to a GPU according to its rank.

- We compare our output against the **single-node reference** implementation.
- With the **OpenMP backend**, SHA-256 checksums over 30 configurations (1–6 workers, sizes 32^3 to 512^3) are **bitwise-identical**.
- With the **CUDA backend**, per-element differences stay within 10^{-15} — near-bitwise agreement, as expected from normal CPU/GPU rounding.

- We use two clusters of the PCAD platform, chosen for their contrasting compute-to-communication balance.
- The *cei* CPU cluster has six nodes (two Xeon Silver 4116 each) on a 10Gbit network.
- The *poti* GPU cluster has five nodes (RTX 4070, 12GB) on a 1Gbps network.
- We trace every run with *akypuera*, convert the traces with *PajeNG*, and analyze them in R.
- We report performance in *MSamples/s*, the metric used for the reference Fletcher code.
- We use *SimGrid SMPI* to simulate 10 and 100Gbps networks beyond our hardware, emulating the real kernel and calibrating against *poti* (error $\leq 15\text{--}17\%$).

Results

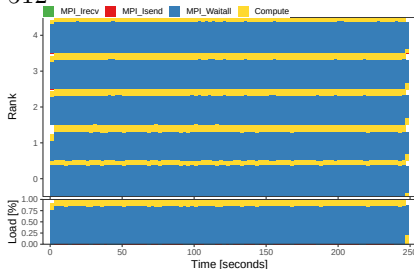
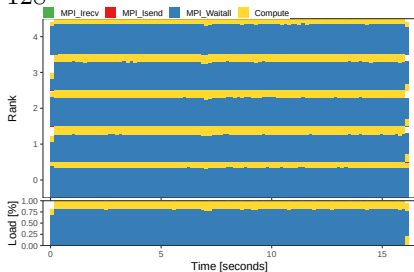
- All charts come from **PajeNG** traces that record each rank's MPI operations over time.
- A synthetic **Compute** state fills the gaps between MPI events, and it includes the PCIe copies to and from the GPU.
- Each experiment is shown two ways: a **Gantt chart** (rank vs. time, one rectangle per event) and a **load chart** (the run split into 100 bins, stacking the Compute and MPI_Waitall fractions).
- The rule for reading them is simple: **a higher compute load means communication is better hidden.**

256³512³

- These runs use the *cei* CPU cluster (10Gbit, OpenMP).
- The compute load is about **75%** at 256³ [top] and rises to about **87%** at 512³ [bottom].
- The Xeon's modest floating-point speed means computation naturally outlasts communication and hides it.
- Larger problems mask even better, because volume grows as $O(n^3)$ while the boundary grows only as $O(n^2)$.

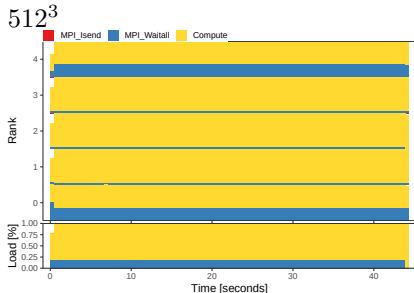
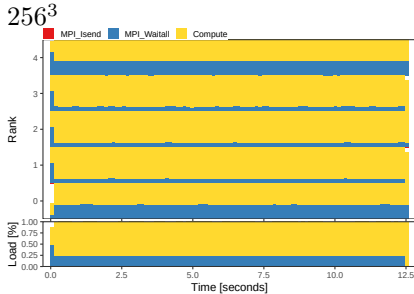
L (Width)	Volume (L^3)	Boundary ($18L^2$)	Ratio
10	1 000	1 800	1.80
100	1 000 000	180 000	0.18
1000	1 000 000 000	18 000 000	0.018

- Making the domain ten times wider yields a thousand times more **useful work** but only a hundred times more **communication**.
- So the communication-to-computation ratio **shrinks toward zero** as each subdomain grows.

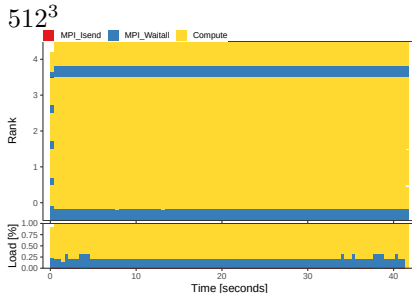
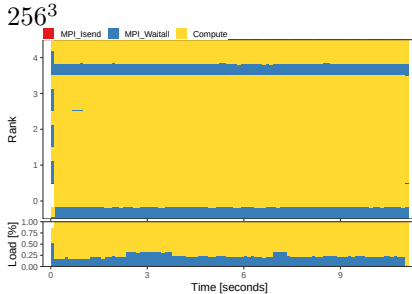


- These runs use the *poti* GPU cluster (1Gbps, CUDA).
- Waitall dominates the timeline, and computation appears only as thin slivers.
- The compute load stays near 20% and does not change from 128^3 [top] to 512^3 [bottom].
- The RTX 4070 finishes faster than the Gigabit link can deliver halos, whatever the problem size.
- Enlarging the domain does not help, and the 12GB of VRAM is itself a ceiling; the only remedy is a faster network.

TRANSITION: 10GBPS SIMULATED NETWORK



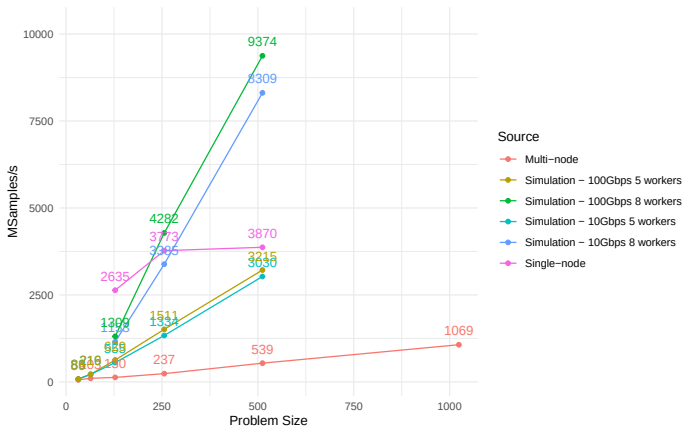
- Here we keep the *poti* model but raise the network to 10Gbps.
- The picture *reverses*: computation now dominates each iteration.
- The compute load reaches about 65–70% at 256³ [top] and 75–80% at 512³ [bottom].
- Waitall shrinks to a minor part of the run.



- Raising the network to **100Gbps** pushes the balance further still.
- The compute load reaches about **85–90%** at 256³ [top] and **exceeds 90%** at 512³ [bottom].
- The ghost-cell exchanges are now almost entirely hidden behind interior computation.
- This confirms the overlap strategy works **once the bandwidth is adequate**.

THROUGHPUT: PARTITIONING TOPOLOGY IS CRITICAL

MPI-BASED 3D ANISOTROPIC RTM
WITH FLETCHER'S METHOD AND
GHOST CELL HALO EXCHANGE



- There is a **dramatic gap** between the 5-worker and 8-worker configurations.
- At 512^3 , 8 workers reach **8309** MSamples/s (10Gbps) and **9374** (100Gbps).
- The 5-worker runs stall at **3030 / 3215**, **below** the single-node baseline of **3870**.
- With 5 workers, distributing the work actually gives **negative speedup**.

WHY? PRIME PROCESS COUNTS

- Because 5 is prime, `MPI_Dims_create` is forced into a $\{5, 1, 1\}$ **slice** layout, whereas 8 factors cleanly into $\{2, 2, 2\}$ **blocks**.
- For a 512^3 domain with stencil thickness 4, the 5-worker slices send $\approx 8\text{MB}$ faces, about **64MB per iteration**.
- The 8-worker blocks send $\approx 2\text{MB}$ faces plus small edges and corners, only about **25MB per iteration**.
- Blocks therefore cut the per-worker volume by roughly **2.5 $\times$** , even though each worker now talks to 7 neighbors instead of 2.
- Raising the bandwidth tenfold barely helps the slice case (+6%), so its real bottleneck is **large-message protocol paths and fixed per-transfer costs**, not raw bandwidth.
- In short, **a prime process count hurts as much as a slow network**.

SUMMARY OF COMPUTE LOAD

Backend	Size	Network	Compute load
CPU (OpenMP)	256^3	10Gbps (real)	$\approx 75\%$
CPU (OpenMP)	512^3	10Gbps (real)	$\approx 87\%$
GPU (CUDA)	128^3	1Gbps (real)	$\approx 20\text{--}25\%$
GPU (CUDA)	256^3	1Gbps (real)	$\approx 20\%$
GPU (CUDA)	512^3	1Gbps (real)	$\approx 20\%$
GPU (CUDA)	256^3	10Gbps (simulated)	$\approx 65\text{--}70\%$
GPU (CUDA)	512^3	10Gbps (simulated)	$\approx 75\text{--}80\%$
GPU (CUDA)	256^3	100Gbps (simulated)	$\approx 85\text{--}90\%$
GPU (CUDA)	512^3	100Gbps (simulated)	$>90\%$

Conclusion

- We observed two sharply contrasting regimes.
- On the **CPU**, modest floating-point speed gives near-perfect masking, reaching 87% compute load at 512^3 .
- On the **GPU**, the RTX 4070 makes the 1Gbps network the bottleneck, holding the compute load near **20%** regardless of problem size.
- Calibrated SimGrid reveals a **smooth transition** as bandwidth grows: $\approx 20\%$ at 1Gbps, $\approx 75\text{--}80\%$ at 10Gbps, and $\approx 85\text{--}90\%$ at 100Gbps.
- The main lesson is that **bandwidth and partitioning topology together determine** how well ghost-cell communication is hidden.
- Prime process counts force slice layouts and can even produce **negative speedup**, so they should be avoided.

- We plan to run on the **Santos Dumont** supercomputer, which offers a high-speed interconnect.
- We will simulate its network topology in detail to support capacity-planning studies.
- We intend to add **HIP kernels** so the code also targets AMD GPUs.
- Finally, we want to compare **CUDA- and HIP-aware MPI** against conventional non-RDMA implementations from the ghost-cell perspective.

Acknowledgments: All experiments in this work used the PCAD platform (<http://pcad.inf.ufrgs.br>) at INF/UFRGS. This work was partially supported by Petrobras through an agreement with UFRGS under project 2020/00182-5, and by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES), Brazil — Finance Code 001.

Special thanks to Marcio Castro (UFSC) for kindly presenting this work on our behalf.

Vinícius Daniel Spadotto, Lucas Mello Schnorr

Institute of Informatics — UFRGS, Porto Alegre, Brazil

